SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – TRANSFORMER MECHANICS WORKSHEET

Course Code:	CSA6506	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO1 – Precision (BL1, BL2)	**Assessment:**	Assessment Tool 3 – Transformer Mechanics Worksheet
Weightage:	40% of CIA	**Total Marks:**	100
CO1 Statement:	*Explain the fundamentals of Artificial Intelligence, Generative AI, Foundation Models, Transformer architecture, tokenization, embeddings, and Large Language Models. (BL1, BL2)*		
Student Name:	ALLAPAREDDY JAHNAVI	**Reg. No.:**	192472214
Section / Batch:	CSE AI/2024	**Date:**	

Instructions to Students

• Answer all questions. It carries 100 Marks. Total: 100 Marks.

· Tokenize a sentence.
· Convert tokens to token IDs.
· Explain embeddings.
· Label a Transformer encoder.
· Identify Query, Key, and Value.
· Explain which words receive higher attention.
· Compare Transformer models.
· Complete a Transformer pipeline.
· Draw a block diagram.

Code of Conduct

I ALLAPAREDDY JAHNAVI (192472214) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

TRANSFORMER MECHANICS WORKSHEET

Question 1: Sentence Tokenization

Consider the sentence: “Transformers enable intelligent text generation.”

Split the sentence into tokens using a suitable tokenizer. Explain how the tokenization may differ between a word-level tokenizer and a subword tokenizer.

Topic: Tokenization | Bloom’s level: Understand

Question 2: Token-to-ID Conversion

Given the token sequence `["AI", "models", "learn"]` and the vocabulary mapping `{"AI": 12, "models": 25, "learn": 8}`, convert the tokens into their corresponding token IDs. What happens if a token is not present in the vocabulary?

Topic: Token IDs | Bloom’s level: Apply

Question 3: Understanding Embeddings

Explain how token IDs are converted into embedding vectors in a Transformer. Why are embedding vectors more useful for language processing than token IDs alone?

Topic: Token embeddings | Bloom’s level: Understand

Question 4: Transformer Encoder Labelling

Draw a Transformer encoder block and label the following components in the correct order: multi-head self-attention, input embeddings with positional information, feed-forward neural network, residual connections, and layer normalization.

Topic: Transformer encoder architecture | Bloom’s level: Apply

Question 5: Query, Key, and Value Identification

In the sentence “The student submitted the assignment because it was complete,” explain how Query, Key, and Value vectors are generated from token representations. Describe their respective roles in determining relationships between words.

Topic: Query–Key–Value | Bloom’s level: Analyze

Question 6: Attention Weight Analysis

Consider the sentence: “The cat chased the mouse because it moved quickly.” Which words might receive higher attention when processing the word “it”? Justify your answer and explain how attention weights help a Transformer capture contextual relationships.

Topic: Self-attention | Bloom’s level: Analyze

Question 7: Comparing Transformer Models

Compare BERT, GPT, and T5 in terms of their attention approach, input-output behaviour, and typical applications. Identify which model would be most suitable for (a) text classification, (b) open-ended text generation, and (c) text summarization, with justification.

Topic: Transformer model comparison | Bloom's level: Analyze

Question 8: Completing the Transformer Pipeline

Complete the following Transformer processing pipeline by filling in the missing stages:

Text Input → _____ → _____ → Embedding and Positional Information → _____ → Output Probabilities → Predicted Token.

Explain the function of each missing stage.

Topic: Transformer pipeline | Bloom's level: Apply

Question 9: Transformer Block Diagram

Draw a block diagram illustrating how an input sentence is processed by a Transformer-based language model to generate an output. Include tokenization, token ID conversion, embeddings, attention layers, feed-forward layers, output projection, and token prediction.

Topic: Transformer architecture | Bloom's level: Create

Question 10: Integrated Transformer Mechanics

A language model receives the sentence "Generative AI creates new content." Trace the sentence through tokenization, token ID conversion, embedding generation, self-attention, contextual representation, and output prediction. Explain how the attention mechanism helps the model understand relationships among the words.

Topic: Integrated Transformer pipeline | Bloom's level: Analyze

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Mechanics Explanation	30%	All key concepts (AI, ML, DL, GenAI, LLMs) correctly identified with clear hierarchical relationships	Most concepts correct; minor gaps in relationships	Several concepts missing or misclassified	Major conceptual errors; map incomplete
Worked-Example Accuracy	25%	Real-world examples linked accurately to each concept	Examples mostly relevant	Few or generic examples	No real-world linkage shown
Mathematical Reasoning	20%	Logical, well-organized structure with clear connectors	Mostly organized; minor structural issues	Some disorganization affecting clarity	Poorly structured, hard to follow
Diagram & Working Clarity	15%	Visually clear, creative, easy to interpret	Neat and readable	Cluttered or inconsistent formatting	Untidy, difficult to interpret
Step Justification	10%	Concise labels/notes that add clarity	Adequate labeling	Minimal or unclear labeling	No supporting labels or explanation

Marks Evaluation:

Mechanics Explanation (30%)	Worked-Example Accuracy(25%)	Mathematical Reasoning (20%)	Diagram & Working Clarity (15%)	Step Justification (10%)	Total Marks (100)

SOLUTION:

TRANSFORMER MECHANICS WORKSHEET

QUESTION 1: Tokenization

A tokenizer converts text into smaller units called tokens. The tokens are then mapped to numerical IDs so that a Transformer can process them.

Word-level tokenization (illustrative): ["Transformers", "enable", "intelligent", "text", "generation", "."]

Subword tokenization (illustrative): ["Transform", "##ers", "enable", "intelligent", "text", "generation", "."].

The exact split depends on the tokenizer vocabulary and model.

Word-level tokenizer	**Subword tokenizer**
Treats each word as a unit.	Can divide words into smaller units.
May require a very large vocabulary.	Handles unfamiliar words using known subwords.
Unseen words may become unknown tokens.	Often represents unseen words through subword combinations.

Advantage: Subword tokenization can represent unfamiliar or rare words by splitting them into known smaller units. This reduces the need to store every possible word in the vocabulary and helps handle out-of-vocabulary words.

QUESTION 2: Token IDs and Vocabulary

Each token is mapped to its corresponding integer ID using the vocabulary.

["AI", "models", "learn"] → [12, 25, 8]

If a word is not directly present in the vocabulary, the tokenizer may split it into known subword tokens. Depending on the tokenizer, it may instead use a special unknown-token ID. The exact behavior depends on the tokenizer design.

QUESTION 3: Word Embeddings

Token IDs are discrete labels, not meaningful numerical measurements. For example, ID 25 is not inherently more similar to ID 26 than to ID 8. An embedding layer maps each token ID to a learned vector of real numbers.

Token ID → Embedding lookup → Vector representation

The embedding vector gives the model a trainable representation that can capture useful patterns. Transformers usually combine token embeddings with positional information so the model can represent both token content and sequence order.

QUESTION 4: Transformer Encoder Architecture

A standard Transformer encoder layer contains multi-head self-attention and a feed-forward neural network. Residual connections and layer normalization help training and information flow.

Input embeddings + positional information

↓

Multi-Head Self-Attention

↓

Residual Connection + LayerNorm

↓

Feed-Forward Neural Network

↓

Residual Connection + LayerNorm

↓

Contextual Output

Multi-head self-attention: lets each token gather information from other tokens in the input sequence using multiple attention heads.

Feed-forward network: applies learned transformations independently at each token position.

Residual connections: add a sublayer's input to its output to support information flow and optimization.

Layer normalization: normalizes hidden representations to support stable training.

Note: Some Transformer implementations use pre-layer normalization rather than the post-layer-normalization order shown above.

QUESTION 5: Query, Key, and Value Vectors

For each token representation, the model learns linear projections that produce Query, Key, and Value vectors.

Query (Q): represents what the current token is looking for in other tokens.

Key (K): represents information used to determine how relevant each token is to the query.

Value (V): carries the information that is combined according to the attention weights.

$$\mathbf{Q = XWQ,\ K = XWK,\ V = XWV}$$

$$\mathbf{Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k})V}$$

For example, when processing “it,” attention may assign weight to “the assignment” or another contextually relevant phrase. The actual attention weights are learned by the model and cannot be determined from the sentence alone.

QUESTION 6: Attention Weights and Context

Self-attention allows the representation of “it” to use information from other words in the sentence. In this example, “it” could refer to “the mouse,” although the sentence is ambiguous and a model’s interpretation may depend on broader context.

The following numbers are hypothetical examples only; they are not measured model outputs:

Illustrative attention from “it”:

Token	Illustrative weight
The	0.02
cat	0.08
chased	0.04
the	0.02
mouse	0.55
because	0.10
it	0.05
moved	0.06
quickly	0.08

Total = 1.00

In this illustrative distribution, “mouse” receives the largest attention weight (0.55), so the example suggests that “it” is associated most strongly with “mouse.” Attention weights show information-use patterns; by themselves, they do not guarantee a correct or complete explanation of a model’s reasoning.

QUESTION 7: Comparison of BERT, GPT, and T5

BERT, GPT, and T5 are Transformer-based model families, but they use different architectures and training objectives.

BERT is commonly suitable for classification because its encoder produces contextual representations that can be used by a classification head. GPT is suitable for generation because it predicts tokens autoregressively, using previous tokens as context. T5 is suitable for summarization because its encoder reads the input document and its decoder generates a shorter output sequence. These are typical uses, not strict limitations; appropriately trained versions can support multiple tasks.

Model	Architecture / attention	Typical strengths	Suitable task
BERT	Encoder-only; bidirectional self-attention	Builds contextual representations using left and right context	Text classification
GPT	Decoder-only; causal self-attention	Predicts the next token and generates sequences	Text generation
T5	Encoder–decoder; encoder self-attention and decoder causal/cross-attention	Treats many NLP tasks as text-to-text transformations	Summarization

QUESTION 8: Transformer Text-Generation Pipeline

- Input text: receive the prompt or preceding text.
- Tokenization: split the text into tokens.
- Token IDs: map tokens to vocabulary indices.
- Embeddings and position information: convert IDs into vectors and represent sequence positions.
- Transformer layers: process vectors using attention and feed-forward networks.
- Output projection: map the final hidden representation to scores, called logits, for vocabulary tokens.
- Softmax and token selection: convert logits into probabilities and select or sample the next token according to the decoding strategy.
- Repeat: append the selected token and run the process again until a stopping condition is reached.

Text → Tokenization → Token IDs → Embeddings + Position
→ Transformer Layers → Logits / Probabilities
→ Predicted Next Token → Repeat

- A model does not usually produce an entire response in one step. Autoregressive generation predicts a next token repeatedly, using the growing sequence as context.

QUESTION 9: Transformer Block Diagram for Output Generation

- The following simplified diagram shows a decoder-only autoregressive generation pipeline:

Input Prompt

↓

Tokenizer

↓

Token IDs

↓

Token Embeddings + Positional Information

↓

Transformer Blocks

(Self-Attention + Feed-Forward Network)

↓

Output Projection (Logits)

↓

Softmax / Decoding Strategy

↓

Next Token

↓

Append Token and Repeat

↓

Generated Text

- Tokenizer and token IDs: convert text into the model's numerical input format.
- Embeddings and positional information: represent token content and order.
- Transformer blocks: contextualize tokens through attention and learned transformations.
- Output projection and decoding: produce token scores and choose the next token.
- Repeat: continue until an end token or another stopping rule is reached.

QUESTION 10: Worked Example — "Generative AI creates new content."

Step 1 — Tokenization:

An illustrative token sequence is ["Generative", "AI", "creates", "new", "content", "."]. Actual tokenization varies by model and tokenizer.

Step 2 — Token IDs:

For demonstration only, suppose the token IDs are [101, 205, 309, 412, 518, 102]. These are hypothetical IDs, not guaranteed IDs from a real model vocabulary.

Step 3 — Embeddings:

Each ID is mapped to a learned vector. Positional information is added or otherwise incorporated so that the model can represent the order of the tokens.

Token IDs → Embedding Vectors + Position Information

↓

Transformer Layers

↓

Contextual Token Representations

Step 4 — Self-attention:

Each token can use information from relevant tokens. For example, the representation of "creates" can incorporate information from "Generative AI," "new," and "content." The actual attention pattern depends on the trained model.

Step 5 — Output prediction:

For an autoregressive language model, the final hidden representation is projected to vocabulary logits and converted to probabilities.

Possible next-token probabilities (hypothetical):

Candidate next token	Example probability
efficiently	0.40
automatically	0.30
quickly	0.20
other tokens	0.10
Total	1.00

If the decoding strategy selects the token with the highest probability, the model would choose "." in this example. A real model's token choices and probabilities depend on its parameters, input context, and decoding settings.

Conclusion: The example demonstrates how text is tokenized, converted into vectors, contextualized by attention, and used to predict a next token. The IDs and probabilities above are illustrative rather than actual model results.